

Solventa

Aseguradora digital sobre Finanzas Abiertas y Datos Abiertos

Dirección de Arquitectura

Documento de Arquitectura de Software

Arquitectura de Solventa

Estilo, modelos, patrones de diseño, programa de experimentación y plan de construcción

Versión 1.0

Documento técnico

Equipo de arquitectura

Integrante	Rol
Jazmin Natalia Córdoba Puerto	Gerencia de proyecto, usabilidad y entrega
Juan Esteban Mejía Izasa	Web front, integración de APIs y pagos
Miguel Alejandro Gómez Alarcón	Arquitectura, Open Finance, KYC, rendimiento y seguridad
Angie Natalia Arandio Niño	Dominio, web back, móvil y pruebas

Bogotá D.C. · 30 de agosto de 2026

1. Contexto y atributos de calidad que dirigen el diseño

Solventa es una aseguradora digital nativa en la nube construida sobre Finanzas Abiertas y Datos Abiertos. Su promesa de negocio —cotizar, suscribir, emitir y pagar siniestros de forma casi instantánea, embebiendo el seguro en el punto de necesidad del cliente— impone exigencias que no se resuelven eligiendo bien un framework, sino decidiendo bien la estructura del sistema.

El diseño que se presenta en este documento no parte de una preferencia tecnológica sino de los nueve escenarios de calidad (ASR) definidos para Solventa. Cada decisión estructural que se documenta a continuación existe porque hay al menos un ASR que la exige.

ASR	Atributo	Medida de respuesta	Consecuencia arquitectural
ASR-1.1	Latencia	p95 < 200 ms con 500 req/min	Caché de perfiles, cálculo asíncrono de lo no crítico
ASR-1.2	Latencia bajo degradación	p95 < 200 ms desde caché, 0% de pérdida	Circuit breaker, timeout agresivo, degradación elegante
ASR-2.1	Modificabilidad	Nueva pasarela en < 4 horas-hombre, 0 cambios en el núcleo	Puertos y adaptadores, inversión de dependencias
ASR-2.2	Modificabilidad	Nuevo ramo en ≤ 2 semanas, 0 cambios en el motor de rating	Configuración externalizada, motor parametrizado
ASR-3.1	Disponibilidad	Recuperación < 30 s, ≥ 99,9%	Fallback a fuente primaria, health checks, bulkhead
ASR-3.2	Disponibilidad multi-zona	RTO ≤ 10 min, RPO ≤ 30 s	Despliegue activo-activo multi-AZ, réplica con promoción
ASR-3.3	Continuidad offline	100% de consultas sin conexión, sync ≤ 10 s	Cliente offline-first, almacenamiento local cifrado
ASR-4.1	Confidencialidad	100% TLS 1.3, 0 PII en texto plano	Tokenización de PII, cifrado en tránsito y reposo
ASR-4.2	Integridad	Alerta de alteración no autorizada < 1 s	Firma digital, almacenamiento inmutable, auditoría

Restricciones que acotan el espacio de diseño

- Contenedores obligatorios para todos los microservicios; nada se instala en el sistema anfitrión.
- PostgreSQL como almacenamiento transaccional primario; no se admite una base NoSQL en ese rol.
- Cumplimiento de Ley 1581 de 2012, Decreto 1297 de 2022, Circular Externa 004 de 2024 y PCI-DSS.
- Consentimiento revocable con efecto en menos de 5 minutos.
- Expansión a México, Chile y Perú dentro de 36 meses, lo que obliga a que la regionalización sea configuración y no una bifurcación del código.

2. Modelos de arquitectura

2.1 Estilo de arquitectura

Antes de presentar las vistas conviene declarar el **estilo** que gobierna la solución, porque es la decisión de la que dependen todas las demás. Solventa no adopta un estilo puro sino una **combinación deliberada de tres**, cada uno resolviendo un problema que los otros no resuelven bien.

Estilo	Dónde se aplica	Qué problema resuelve	ASR que lo exige
Microservicios	Núcleo de negocio: siete servicios delimitados por contexto de dominio	Permite escalar y desplegar de forma independiente el servicio con exigencia de latencia, sin arrastrar a los demás	ASR-1.1 · ASR-3.2
Por capas	Edge → BFF → Núcleo → Adaptadores	Confina las responsabilidades transversales —seguridad en el borde, formato por canal en el BFF, integración externa en los adaptadores— para que no se repitan en cada servicio	ASR-2.1 · ASR-4.1
Orientado a eventos	Comunicación asíncrona entre servicios del núcleo vía Event Bus	Saca del camino crítico todo lo que no es necesario para responder al usuario y desacopla productores de consumidores	ASR-1.2 · ASR-2.1

Por qué microservicios y no un monolito modular. Un monolito habría simplificado el desarrollo en el plazo disponible, y esa era una alternativa seria. Se descartó por una razón concreta y no por moda: el servicio de Perfilamiento y Scoring es el único con una exigencia de latencia de 200 ms y el único que enfrenta picos de diez veces el tráfico normal durante campañas de socios. En un monolito, escalar ese componente obliga a replicar todo el sistema, lo que multiplica el costo de infraestructura por una necesidad que solo tiene una parte. La independencia de despliegue no es un fin: es lo que hace económicamente viable el ASR-1.1.

Por qué capas sobre los microservicios. Los microservicios por sí solos no dicen dónde vive la autenticación, ni el formato de respuesta por canal, ni la traducción hacia proveedores externos. Sin capas, cada servicio termina implementando su propia versión de esas tres cosas, que es la principal fuente de inconsistencias de seguridad en arquitecturas de microservicios. Las capas asignan un único lugar a cada responsabilidad transversal.

Por qué eventos además de llamadas sincrónicas. La emisión de una póliza dispara efectos en cuatro servicios. Si Emisión los llamara de forma sincrónica, su latencia sería la suma de las cuatro y su disponibilidad el producto de las cuatro. El estilo orientado a eventos convierte esa suma en una sola escritura al bus. La contrapartida es consistencia eventual, que se acepta para efectos secundarios pero **no** para el cobro de la prima, que permanece sincrónico dentro de la transacción de emisión.

Lo que el estilo deja fuera. No se adopta arquitectura sin servidor para el núcleo, pese a que encajaría con los picos de tráfico, porque los arranques en frío son incompatibles con un presupuesto de 200 ms y porque el equipo no tiene experiencia previa que permita estimar su comportamiento con confianza. Tampoco se adopta una malla de servicios: aportaría observabilidad y control de tráfico, pero su costo de operación no se justifica con siete servicios y un equipo de cuatro personas.

2.2 Vista de contexto

Antes de descomponer el sistema conviene fijar su frontera: quién lo usa, de qué terceros depende y qué intercambia con cada uno.

Diagrama de Contexto — Solventa

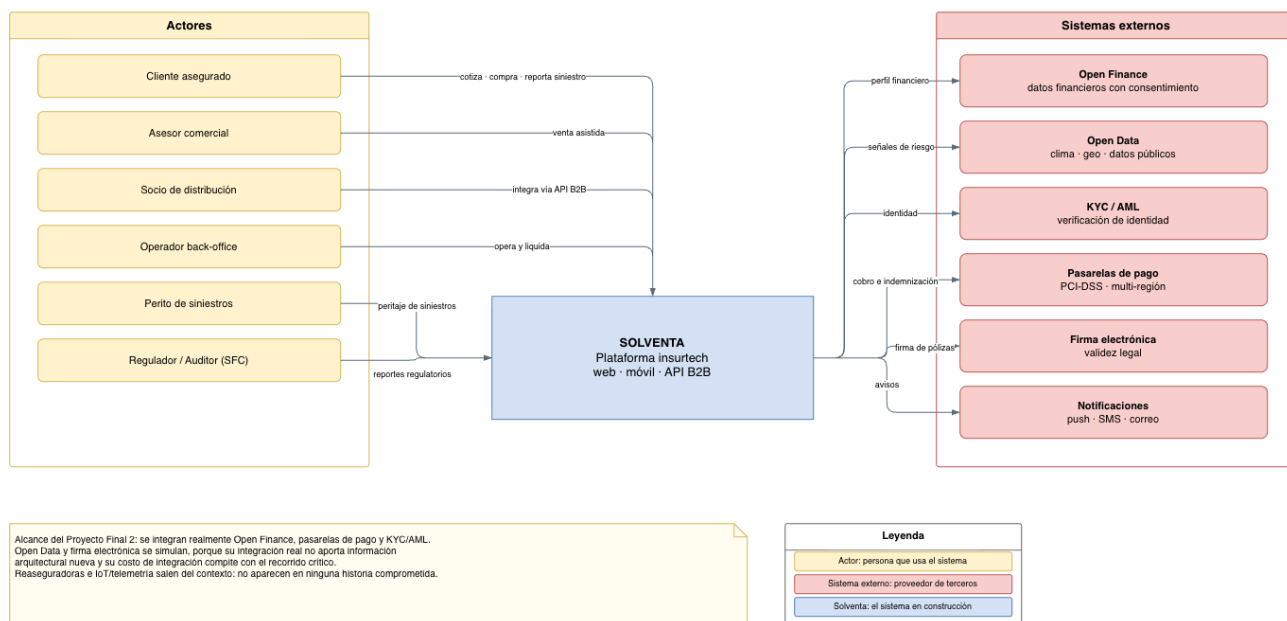


Figura 1. Diagrama de contexto: actores, sistemas externos y qué intercambia Solventa con cada uno

Seis actores. El cliente asegurado es el usuario principal, pero no el único que impone requisitos: el regulador exige reportes trazables, el perito necesita acceso acotado a la evidencia de un siniestro, y el socio de distribución consume la plataforma por API sin pasar nunca por la interfaz. Ese último es el que obliga a que la API B2B sea un canal de primera clase y no un añadido.

Cinco sistemas externos, con distinto grado de compromiso. La distinción importa para el alcance: en la fase de construcción se integran realmente Open Finance, las pasarelas de pago y KYC/AML, porque son los que introducen incertidumbre arquitectural. Open Data y firma electrónica se simulan: su integración real no aporta información de diseño nueva y su costo compete con el recorrido crítico.

Lo que salió del contexto. Las versiones anteriores incluían reaseguradoras e IoT/telemetría como sistemas externos. Se retiran porque no aparecen en ninguna historia del backlog comprometido, y un contexto que muestra integraciones que nadie va a construir describe un sistema que no existe.

2.3 Vista funcional

La vista funcional muestra cómo se descompone el sistema en componentes y cómo colaboran para atender los recorridos de negocio. La estructura es de cuatro capas más una plataforma de datos transversal.

Vista Funcional — Solventa

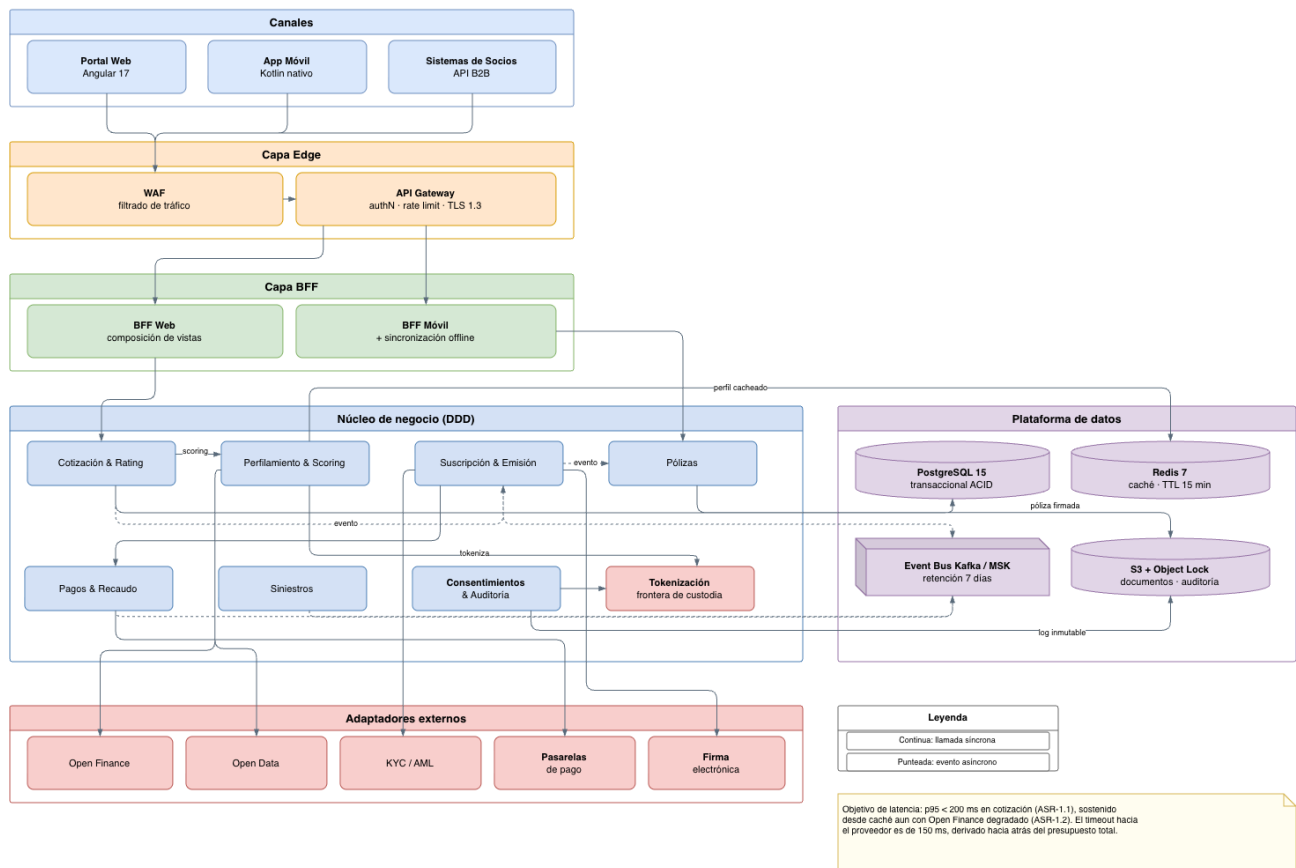


Figura 2. Vista funcional: capas Edge, BFF, núcleo de negocio y adaptadores externos

Capa de canales. Tres consumidores con necesidades distintas: el portal web en Angular, la aplicación móvil nativa en Kotlin y los sistemas de socios distribuidores que consumen la API B2B. Son deliberadamente delgados: no contienen reglas de negocio, porque una regla que vive en el canal debe reimplementarse en cada canal nuevo.

Capa Edge. Concentra lo que debe ocurrir antes de que una petición toque lógica de negocio: filtrado de tráfico malicioso en el WAF, y en el API Gateway la autenticación, la limitación de tasa por cliente y por socio, y la terminación TLS. Situar esto en el borde evita que cada servicio reimplemente controles de seguridad, que es la principal fuente de inconsistencias de seguridad en arquitecturas de microservicios.

Capa BFF. Un backend por canal. El BFF Web compone respuestas orientadas a pantallas amplias con muchos datos por petición; el BFF Móvil compone respuestas compactas y además gestiona el protocolo de sincronización offline. Sin esta capa, o bien el núcleo se contamina con formatos específicos de canal, o bien el móvil paga el costo de recibir cargas útiles pensadas para web.

Núcleo de negocio. Siete servicios delimitados por contexto de dominio: Cotización y Rating, Perfilamiento y Scoring, Suscripción y Emisión, Pólizas, Sinistros, Pagos y Recaudado, y Consentimientos y Auditoría. La frontera de cada servicio coincide con una frontera de lenguaje del negocio, no con una capa técnica. Los servicios se comunican de forma síncrona solo cuando el resultado es necesario para responder al usuario; el resto de la colaboración ocurre por eventos.

Plataforma de datos. PostgreSQL para lo transaccional, Redis como caché de perfiles de riesgo, Kafka como bus de eventos de dominio y almacenamiento de objetos con bloqueo de escritura para documentos y auditoría.

Adaptadores externos. Cada proveedor externo —Open Finance, Open Data, KYC/AML, pasarelas de pago y firma electrónica— se alcanza exclusivamente a través de un adaptador que traduce entre el modelo del proveedor y el modelo de dominio de Solventa. Ningún servicio del núcleo conoce el formato de un proveedor.

El mismo nivel funcional como modelo componente-conector

La vista anterior sirve para orientarse, pero sus flechas no dicen si una interacción es síncrona o asíncrona ni cuál es el contrato que se usa. El modelo siguiente expresa la misma estructura en **notación UML de componente-conector**, donde cada símbolo tiene significado definido.

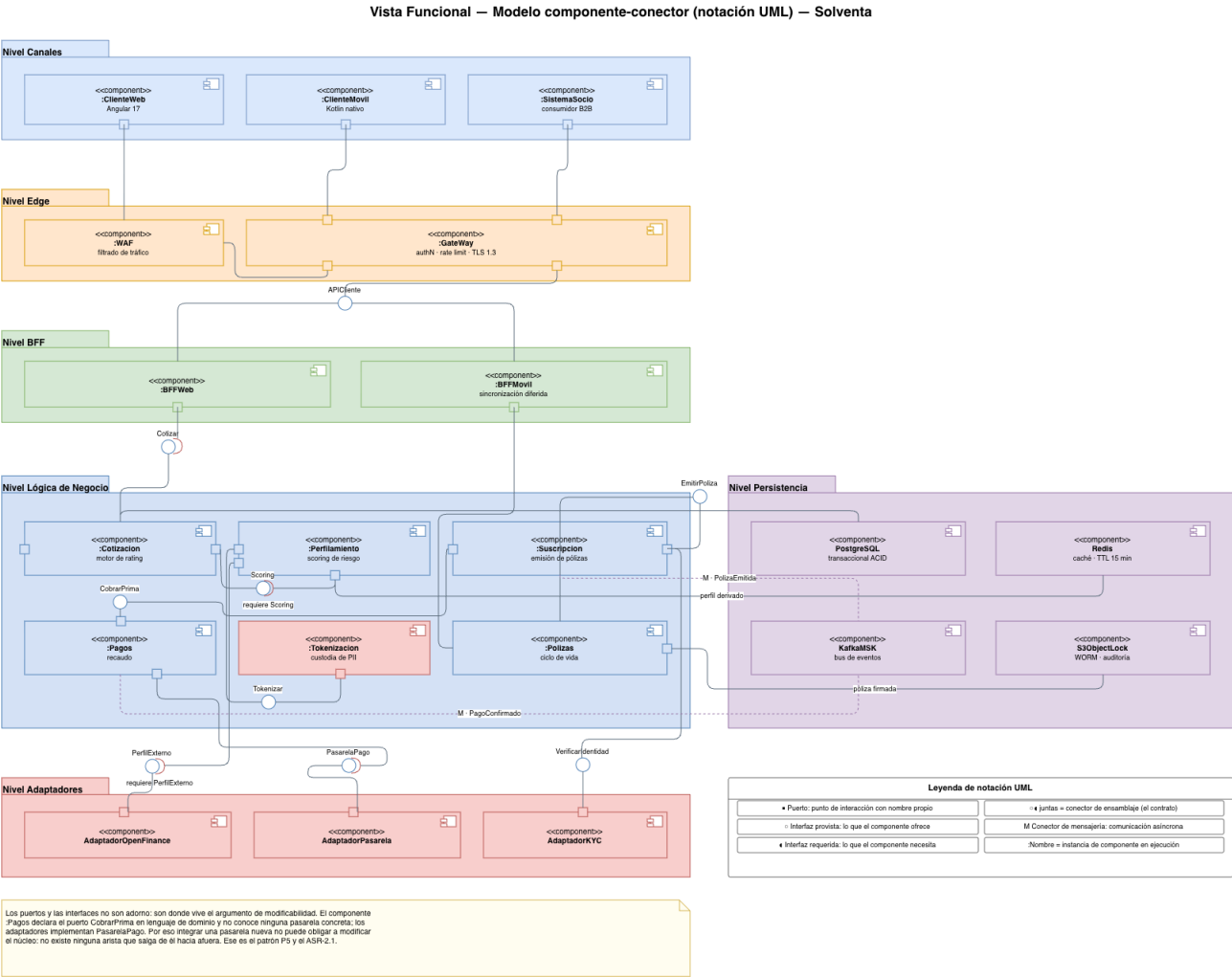


Figura 3. Vista funcional como modelo componente-conector en notación UML, con puertos e interfaces provistas y requeridas

Símbolo	Qué es	Qué comunica
Cuadrado sobre el borde	Puerto	Punto de interacción con nombre propio; aísla el interior del componente de su exterior
Círculo	Interfaz provista	Lo que el componente ofrece
Semicírculo	Interfaz requerida	Lo que el componente necesita
Círculo encajado en semicírculo	Conector de ensamblaje	El contrato concreto entre dos componentes
Conector marcado con M	Conector de mensajería	Comunicación asíncrona por el bus de eventos
Nombre precedido de dos puntos	Instancia de componente	Algo que se ejecuta, no un tipo

Por qué esta notación y no cajas genéricas. Los puertos y las interfaces son exactamente donde vive el argumento de modificabilidad del ASR-2.1. El componente :Pagos declara el puerto CobrarPrima en lenguaje de dominio —autorizar, confirmar, reversar— y no conoce ninguna pasarela concreta; los adaptadores

implementan **PasarelaPago**. Integrar una pasarela nueva no puede obligar a modificar el núcleo porque no existe ninguna arista que salga de él hacia afuera. Con cajas genéricas eso hay que afirmarlo en prosa; en notación UML se lee en el diagrama.

Los ocho contratos del sistema. El modelo nombra las interfaces que constituyen la frontera entre componentes: **APICliente**, **Cotizar**, **Scoring**, **Tokenizar**, **CobrarPrima**, **EmitirPoliza**, **PerfilExterno**, **PasarelaPago** y **VerificarIdentidad**. Esa lista es el contrato que la fase de construcción debe respetar: la revisión de arquitectura validará que el código se conforme a esta arquitectura, y estas interfaces son la parte verificable de ese compromiso.

2.4 Vista de despliegue

La vista de despliegue muestra dónde se ejecuta cada componente y qué mecanismos de redundancia lo protegen.

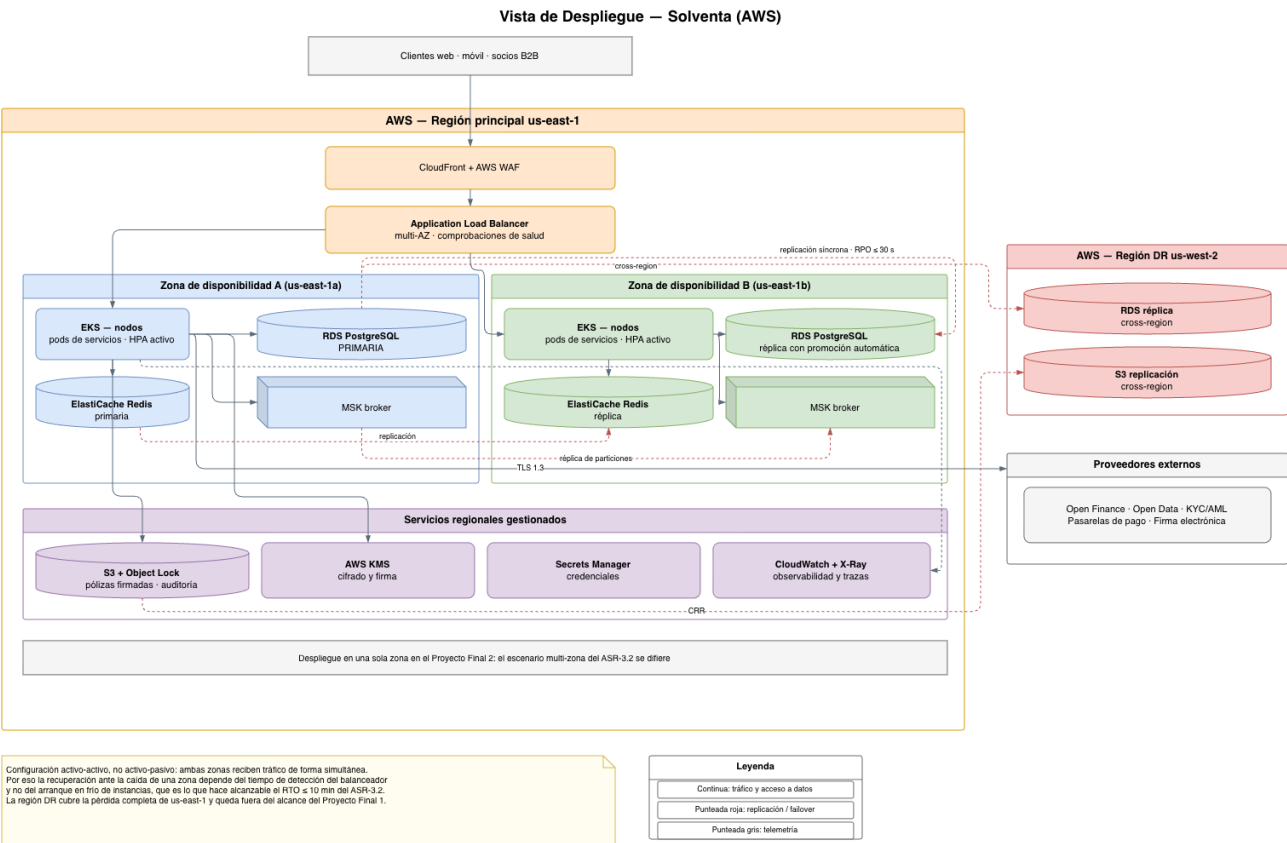


Figura 4. Figura 3. Vista de despliegue en AWS con redundancia multi-zona y región de recuperación

Distribución activo-activo. Los nodos de EKS están repartidos entre dos zonas de disponibilidad y ambas reciben tráfico simultáneamente. No es una configuración activo-pasivo: si una zona cae, la capacidad de la otra ya está caliente y sirviendo peticiones, lo que hace que el tiempo de recuperación dependa del tiempo de detección del balanceador y no del tiempo de arranque de instancias. Esta es la decisión que hace alcanzable el RTO de 10 minutos del ASR-3.2.

Datos. RDS PostgreSQL opera en configuración multi-AZ con réplica en espera y promoción automática. La replicación sincrónica hacia la zona B es lo que sostiene el RPO de 30 segundos. Redis y los brokers de Kafka también están replicados entre zonas.

Región de recuperación. Una réplica de lectura entre regiones hacia us-west-2 y replicación entre regiones del almacenamiento de objetos cubren el escenario de pérdida completa de la región principal. Este escenario está fuera del alcance de los experimentos de la fase de diseño, pero se documenta porque condiciona decisiones de diseño que sí se toman ahora, como no depender de identificadores generados localmente por instancia.

Servicios regionales. El servicio de gestión de llaves cifra los datos en reposo y firma las pólizas emitidas; el almacenamiento de objetos con bloqueo de escritura garantiza que un registro de auditoría no pueda alterarse ni siquiera con credenciales administrativas; la observabilidad centralizada recoge métricas y trazas distribuidas de todos los servicios.

2.5 Vista de información

La vista de información muestra las entidades de dominio, cómo se clasifican sus datos según sensibilidad y dónde se persiste cada clase.

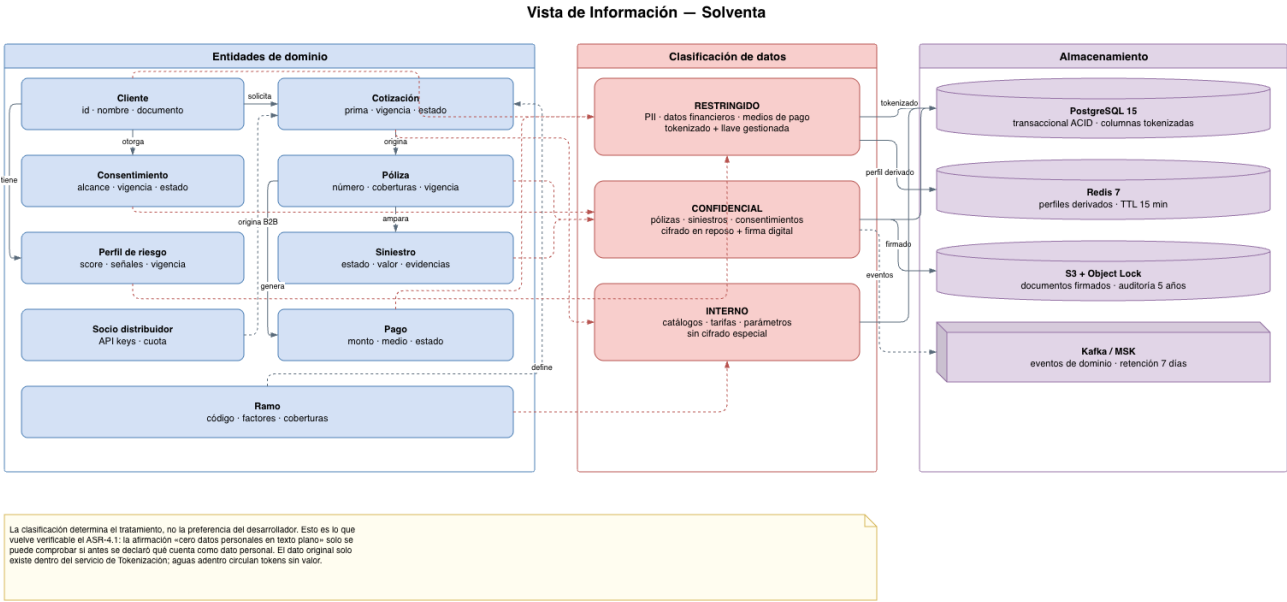


Figura 5. Figura 4. Vista de información: entidades, clasificación de datos y almacenamiento

Entidades de dominio. El cliente es la entidad raíz. De él dependen su perfil de riesgo, sus consentimientos y sus cotizaciones. Una cotización origina una póliza; una póliza ampara siniestros y genera pagos. Los socios distribuidores originan cotizaciones por el canal B2B, lo que significa que el modelo debe soportar una cotización sin cliente registrado previamente.

Clasificación de datos. El sistema clasifica los datos en tres niveles y el nivel determina el tratamiento, no la preferencia del desarrollador:

Nivel	Contenido	Tratamiento
Restringido	Información personal identificable, datos financieros de Open Finance, datos de medios de pago	Tokenizado antes de persistirse; el dato original solo existe en el servicio de tokenización, cifrado con llave gestionada
Confidencial	Pólizas, siniestros, consentimientos	Cifrado en reposo; los documentos además firmados digitalmente
Interno	Catálogos de ramos, tarifas, parámetros de configuración	Sin cifrado especial; su exposición no genera daño

Esta clasificación es la que hace verificable el ASR-4.1: la afirmación «cero datos personales en texto plano» solo se puede comprobar si antes se declaró qué cuenta como dato personal.

Modelo de dominio detallado

La vista anterior resume las entidades para poder relacionarlas con su clasificación. El modelo siguiente las desarrolla con sus atributos y cardinalidades, que es lo que se traduce en esquema de base de datos.

Modelo de Dominio — Solventa

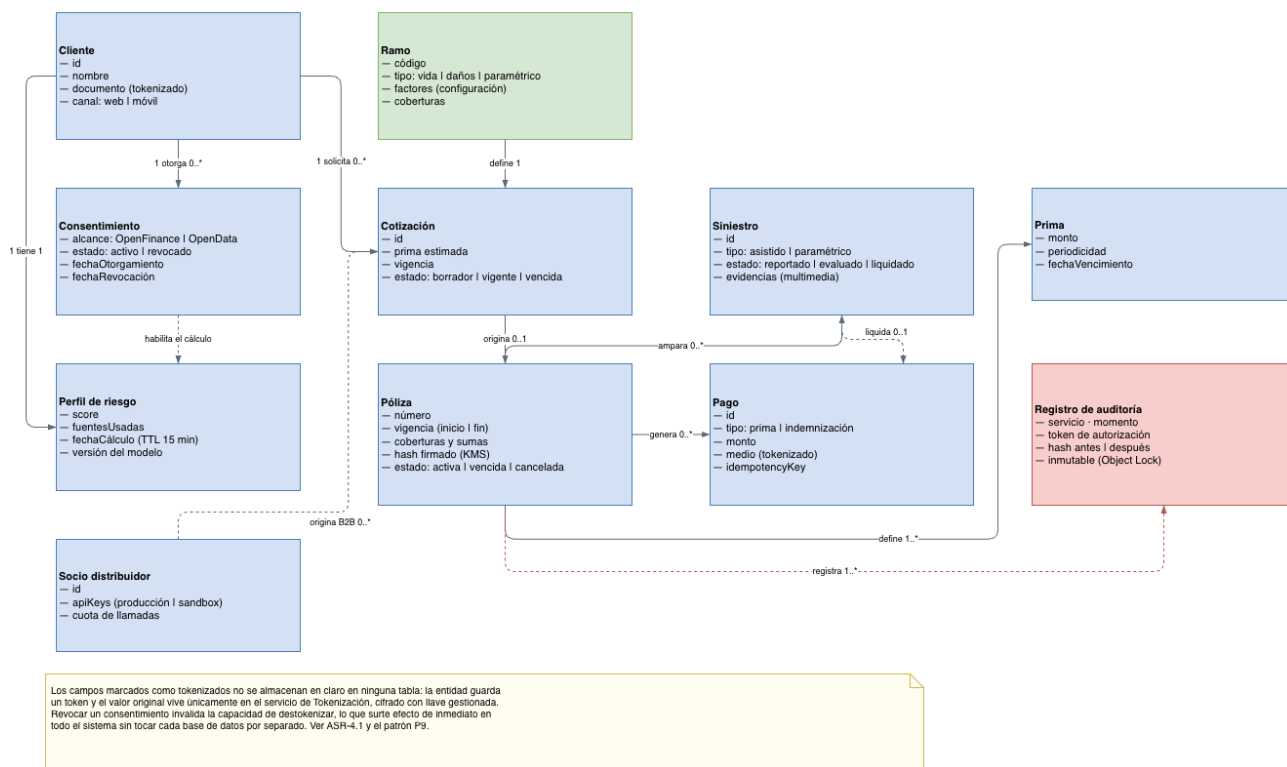


Figura 6. Figura 5. Modelo de dominio: entidades, atributos y cardinalidades

Tres decisiones del modelo merecen explicación:

Los campos marcados como tokenizados no se almacenan en claro en ninguna tabla. La entidad guarda un token y el valor original vive únicamente en el servicio de Tokenización. Por eso **Cliente.documento** y **Pago.medio** aparecen anotados: son los dos puntos por donde entra información personal al modelo.

El registro de auditoría es una entidad de primera clase, no un efecto secundario. Aparece con sus atributos y su relación con Póliza porque el ASR-4.2 exige que sea inmutable y retenido cinco años; tratarlo como una tabla de log accesoria llevaría a diseñarlo sin esa garantía.

Ramo es una entidad de configuración, no de código. Sus factores y coberturas se cargan desde configuración versionada, que es lo que sostiene el ASR-2.2: agregar un ramo nuevo es cargar una definición, no modificar el motor de rating.

Persistencia. PostgreSQL guarda el estado transaccional; Redis guarda perfiles de riesgo con vencimiento de 15 minutos, lo que acota la ventana de exposición de datos derivados; el almacenamiento de objetos guarda documentos firmados y el registro de auditoría en modo de solo escritura; Kafka retiene los eventos de dominio 7 días para permitir su reproceso.

3. Diseño detallado — patrones aplicados y su razonamiento

Esta sección documenta cada patrón estructural adoptado. Para cada uno se indica el problema concreto que resuelve, el ASR que lo motiva, cómo se aplica en Solventa y qué se sacrifica al adoptarlo. Un patrón sin contrapartida declarada suele ser un patrón que no se entendió.

3.1 Tabla resumen

#	Patrón	ASR que atiende	Componente donde se aplica
---	--------	-----------------	----------------------------

#	Patrón	ASR que atiende	Componente donde se aplica
P1	Caché de perfiles con lectura anticipada	ASR-1.1, ASR-1.2	Perfilamiento y Scoring · Redis
P2	Interruptor de circuito	ASR-1.2, ASR-3.1	Adaptadores externos
P3	Tiempo de espera acotado con reintento y espera creciente	ASR-1.2	Adaptadores externos
P4	Mamparo de aislamiento	ASR-3.1	Núcleo de negocio
P5	Puertos y adaptadores	ASR-2.1	Adaptadores externos · núcleo
P6	Estrategia con configuración externalizada	ASR-2.2	Cotización y Rating
P7	Backend por canal	ASR-3.3	Capa BFF
P8	Publicación y suscripción de eventos de dominio	ASR-1.2, ASR-2.1	Event Bus
P9	Tokenización de datos sensibles	ASR-4.1	Consentimientos · Perfilamiento
P10	Almacenamiento inmutable con firma digital	ASR-4.2	Pólizas · auditoría
P11	Cliente con prioridad al modo desconectado	ASR-3.3	App móvil · BFF Móvil
P12	Autoescalado con comprobaciones de salud	ASR-3.2	EKS

3.3 Vista de asignación de patrones

Esta vista muestra dónde vive cada patrón dentro de la arquitectura. Las anotaciones entre llaves indican el patrón que gobierna ese componente o esa capa.

Asignación de patrones sobre la arquitectura — Solventa

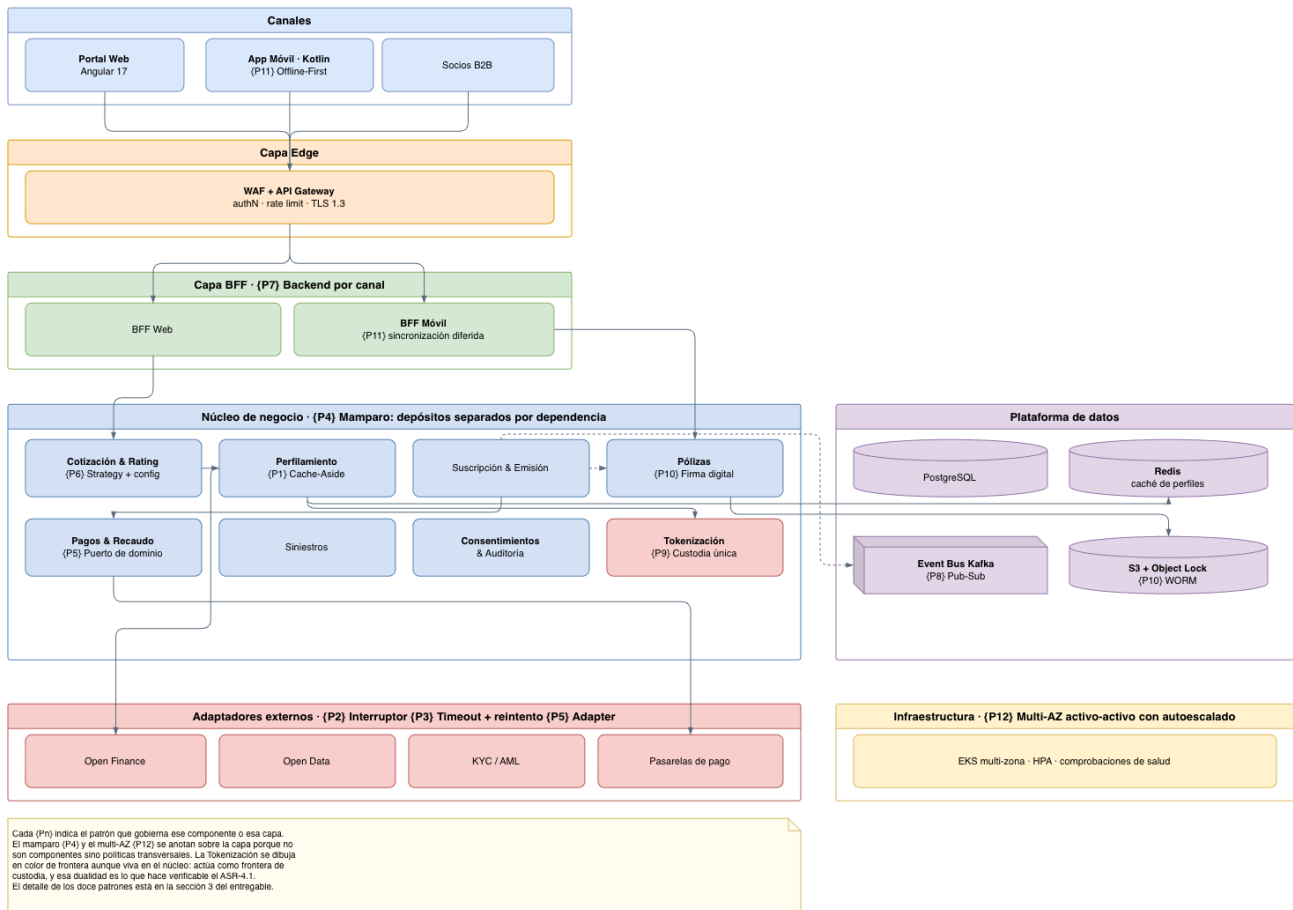


Figura 7. Figura 6. Asignación de los doce patrones sobre los componentes de la arquitectura

Tres observaciones sobre la asignación:

1. **Los patrones de disponibilidad se concentran en la frontera con lo externo.** El interruptor de circuito, el tiempo de espera acotado y el adaptador viven todos en la capa de adaptadores, porque es ahí donde el sistema deja de controlar lo que ocurre. Ningún servicio del núcleo implementa resiliencia frente a terceros: la hereda de la frontera.
2. **El mamparo es transversal al núcleo y no un componente.** No aparece como una caja porque no lo es: es la política de que cada servicio tenga depósitos de recursos separados por dependencia. Se representa como propiedad de la capa.
3. **La tokenización es el único componente del núcleo dibujado en color de frontera.** Es deliberado: aunque se ejecuta dentro del núcleo, actúa como frontera de custodia, y esa dualidad es lo que hace verificable el ASR-4.1.

3.4 Matriz de trazabilidad

Cada fila se lee así: el patrón materializa esas tácticas, vive en esos componentes, existe para satisfacer ese ASR, y ese experimento comprueba si lo consigue.

Patrón	Tácticas que materializa	Componentes donde vive	ASR	Experimento
P1 Cache-Aside	Mantener múltiples copias de datos computados · Reducir la demanda computacional en el	Perfilamiento · Redis	ASR-1.1 · ASR-1.2	EXP-01

Patrón	Tácticas que materializa	Componentes donde vive	ASR	Experimento
	camino crítico			
P2 Interruptor de circuito	Detectar fallas por tasa de respuesta · Degradación elegante	Adaptadores externos	ASR-1.2 · ASR-3.1	EXP-02
P3 Tiempo de espera con reintento	Acotar el tiempo de espera · Reintento de operaciones idempotentes	Adaptadores externos	ASR-1.2	EXP-02
P4 Mamparo de aislamiento	Contener el fallo limitando recursos compartidos	Todo el núcleo (depósitos por dependencia)	ASR-3.1	EXP-02 (instrumentado)
P5 Puertos y adaptadores	Encapsular la variabilidad · Invertir dependencias · Diferir el enlace	Pagos & Recaudo · capa de adaptadores	ASR-2.1	EXP-03
P6 Strategy con configuración externalizada	Encapsular la variabilidad · Diferir el enlace al despliegue	Cotización & Rating · configuración de ramos	ASR-2.2	EXP-03
P7 Backend por canal	Aislar la variabilidad del canal · Reducir el acoplamiento entre canales	BFF Web · BFF Móvil	ASR-3.3	Prototipo móvil
P8 Publicación y suscripción	Sacar del camino crítico lo no esencial · Desacoplar productor y consumidor	Event Bus · servicios del núcleo	ASR-1.2 · ASR-2.1	EXP-02
P9 Tokenización	Limitar el acceso por custodia centralizada · Cifrar en tránsito y reposo · Auditar cada acceso	Tokenización · Perfilamiento · Consentimientos	ASR-4.1	EXP-03
P10 Almacenamiento inmutable con firma	Verificar integridad · Registro no repudiable · Autorizar por token de servicio	Pólizas · Object Storage · Auditoría	ASR-4.2	Prueba de configuración
P11 Cliente offline-first	Mantener copia local · Sincronización diferida · Acotar la vigencia del dato replicado	App móvil · BFF Móvil	ASR-3.3	Prototipo móvil
P12 Multi-AZ con autoescalado	Redundancia activa · Replicación síncrona · Detección por comprobación de salud	EKS · RDS · MSK	ASR-3.2	fase de construcción

Cobertura. Los nueve ASR quedan cubiertos por al menos un patrón, y los doce patrones tienen al menos un ASR que los justifica. No hay patrones huérfanos —adoptados sin un requisito que los exija— ni ASR sin mecanismo asignado.

3.5 Estructura de los patrones críticos

Los tres patrones siguientes se detallan estructuralmente porque son los que sostienen los ASR de mayor exigencia y los que más fácilmente se implementan mal.

El camino de latencia: P1 + P2 + P3 + P4 operando juntos

Los cuatro patrones del camino crítico no actúan por separado sino como una cadena de decisiones. Este diagrama muestra el recorrido completo de una solicitud de cotización y dónde interviene cada uno.

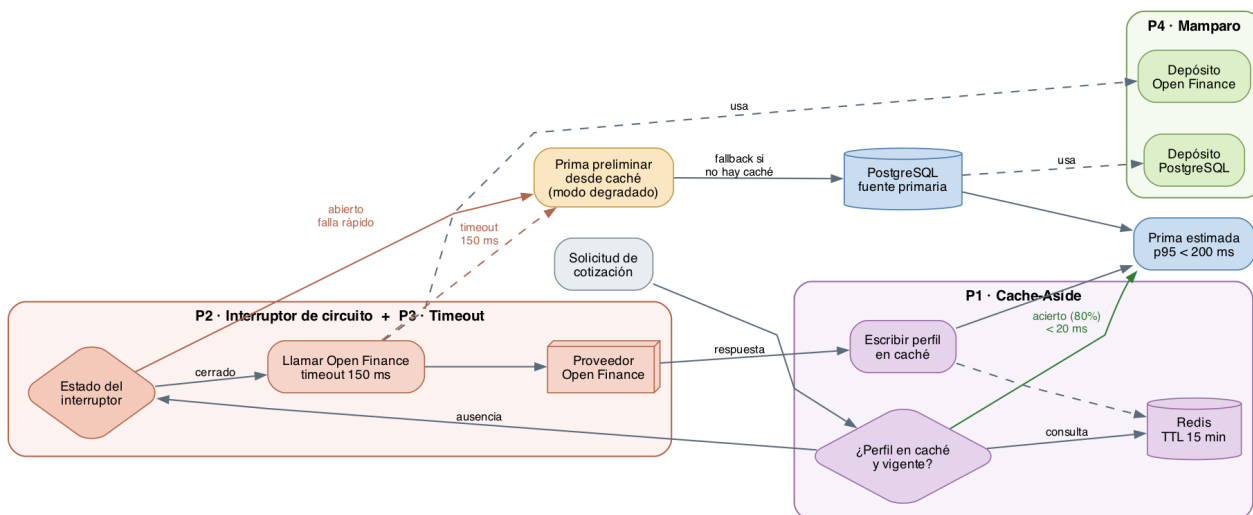


Figura 8. Figura 7. Composición de los patrones Cache-Aside, Interruptor de circuito, Timeout y Mamparo en el camino de cotización

La lectura importante es que **hay tres salidas distintas hacia una respuesta válida** y ninguna es un error: acierto de caché en menos de 20 ms, respuesta del proveedor dentro del presupuesto de 150 ms, o prima preliminar en modo degradado. El sistema nunca se queda esperando, que es precisamente lo que exige la medida de «0% de solicitudes perdidas» del ASR-1.2.

La modificabilidad: P5 + P6 y la dirección de las dependencias

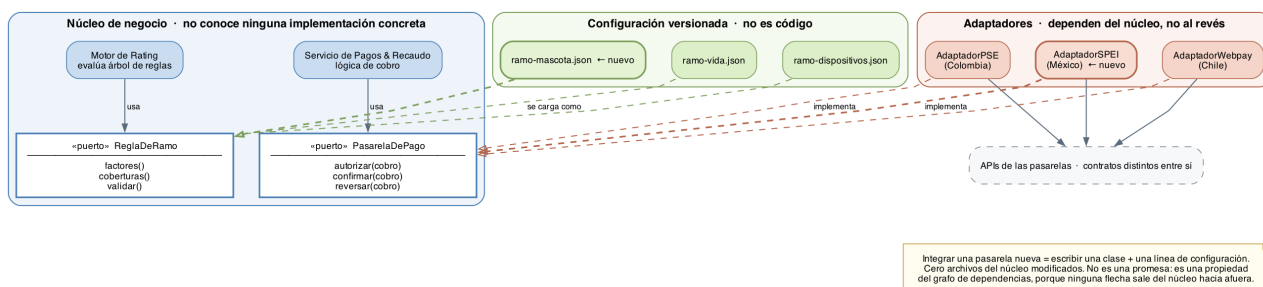


Figura 9. Figura 8. Puertos, adaptadores y configuración externalizada: ninguna flecha sale del núcleo

Lo relevante de este diagrama no son las cajas sino **el sentido de las flechas**. El núcleo define los puertos `PasarelaDePago` y `ReglaDeRamo` y no depende de nada externo; los adaptadores y los archivos de configuración apuntan *hacia* el núcleo. Por eso la afirmación del ASR-2.1 —«cero cambios en servicios del núcleo»— no es una promesa de disciplina del equipo sino una propiedad estructural: agregar `AdaptadorSPEI` no puede obligar a modificar el núcleo porque no existe ninguna arista que vaya en esa dirección.

La confidencialidad: P9 y la frontera de custodia

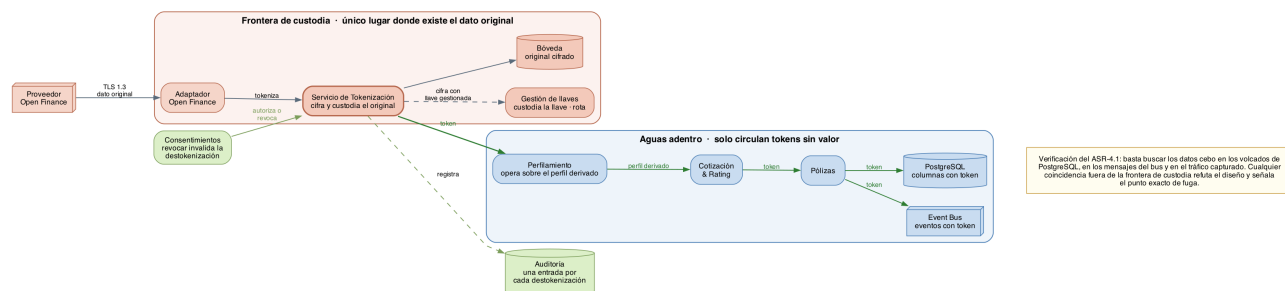


Figura 10. Figura 9. Tokenización con custodia única: aguas adentro solo circulan tokens

El diagrama define una **frontera de custodia**. A su izquierda existe el dato original; a su derecha solo existen tokens, que no tienen valor si se filtran. Esa frontera es lo que convierte el ASR-4.1 en algo verificable: para comprobar que no hay datos personales en texto plano basta con inspeccionar todo lo que hay a la derecha y comprobar que solo aparecen tokens. Sin la frontera, la verificación tendría que ser exhaustiva sobre los siete servicios del núcleo.

Además, **revocar un consentimiento se reduce a invalidar la capacidad de destokenizar**, lo que surte efecto de inmediato en todo el sistema. Con el dato replicado en cada servicio, cumplir el plazo regulatorio de 5 minutos exigiría borrarlo en siete bases de datos y demostrarlo.

3.6 P1 — Caché de perfiles con lectura anticipada

Problema. El cálculo del perfil de riesgo requiere consultar Open Finance y Open Data. Esas consultas tardan cientos de milisegundos en el mejor caso. Si la cotización espera esa consulta de forma sincrónica, el objetivo de 200 ms del ASR-1.1 es inalcanzable por construcción: ninguna optimización interna compensa una llamada de red externa a un tercero.

Aplicación. El servicio de Perfilamiento consulta primero Redis. Si el perfil está presente y vigente, responde desde caché. Si no, consulta al proveedor externo, responde y almacena el resultado con vencimiento de 15 minutos. Para los clientes que llegan por un socio distribuidor con campaña programada, el perfil se precaculcula en lote antes de la campaña.

Razonamiento frente al ASR. El ASR-1.1 exige p95 menor a 200 ms con 500 peticiones por minuto. Con una tasa de acierto de caché del 80% —conservadora para una ventana de 15 minutos en un flujo de cotización donde el usuario itera sobre planes— el percentil 95 queda determinado por el camino de caché y no por el camino externo. El ASR-1.2 va más lejos: exige que bajo degradación del proveedor el sistema siga respondiendo en 200 ms, lo que solo es posible si el perfil cacheado puede servir como respuesta válida y no solo como optimización.

Contrapartida aceptada. Un perfil cacheado puede estar hasta 15 minutos desactualizado. Para un scoring de seguro esto es tolerable: la situación financiera de una persona no cambia de forma material en ese intervalo. No sería tolerable para un saldo de cuenta, y por eso el caché se aplica al perfil derivado y nunca al dato financiero crudo.

3.7 P2 — Interruptor de circuito

Problema. Cuando un proveedor externo se degrada, el modo de falla más dañino no es que responda con error, sino que responda lento. Las peticiones se acumulan, los hilos del servicio se agotan esperando, y un problema de un proveedor se convierte en la caída completa de Solventa.

Aplicación. Cada adaptador externo está envuelto en un interruptor con tres estados. En estado cerrado las peticiones fluyen normalmente. Cuando la proporción de fallos o de respuestas por encima del umbral de tiempo supera el 50% en una ventana móvil, el interruptor abre y las peticiones siguientes fallan de inmediato sin tocar la red, activando la ruta de degradación. Tras un intervalo, el interruptor pasa a semiabierto y deja pasar un número reducido de peticiones de prueba.

Razonamiento frente al ASR. El ASR-1.2 describe exactamente este escenario: 5.000 peticiones concurrentes con el proveedor respondiendo por encima de 700 ms, y exige 0% de peticiones perdidas. Sin interruptor, las peticiones se pierden por agotamiento de recursos. Con interruptor, la petición no se pierde: se atiende por la ruta degradada usando el perfil cacheado. La diferencia entre cumplir y no cumplir el ASR-1.2 no está en la velocidad del sistema sino en su capacidad de dejar de esperar a tiempo.

Contrapartida aceptada. Durante la apertura del interruptor, clientes cuyo perfil no está en caché reciben una cotización preliminar en lugar de una definitiva. Se prefiere una respuesta aproximada e inmediata sobre una respuesta exacta que no llega.

3.8 P3 — Tiempo de espera acotado con reintento y espera creciente

Problema. Un tiempo de espera mal calibrado anula el interruptor de circuito. Si el tiempo de espera del cliente supera el del usuario, el usuario abandona antes de que el sistema decida.

Aplicación. Cada llamada externa tiene un tiempo de espera explícito, derivado hacia atrás desde el presupuesto de latencia total: si el objetivo extremo a extremo es 200 ms, el adaptador de Open Finance no puede tener un tiempo de espera de 3 segundos. Los reintentos se aplican solo a operaciones idempotentes, con espera creciente y un componente aleatorio para no sincronizar a todos los clientes en la misma reintentada.

Razonamiento frente al ASR. Es el patrón que hace operativo el presupuesto de latencia del ASR-1.1. Un presupuesto de latencia que no se traduce en tiempos de espera configurados es una intención, no un mecanismo.

Contrapartida aceptada. Tiempos de espera agresivos aumentan la proporción de respuestas degradadas cuando la red está lenta pero funcional. Se acepta porque el ASR prioriza responder a tiempo sobre responder con el dato más fresco.

3.9 P4 — Mamparo de aislamiento

Problema. Servicios que comparten un mismo depósito de conexiones o de hilos se hunden juntos. Si Siniestros agota el depósito de conexiones a la base de datos, Cotización deja de responder aunque no tenga ningún problema propio.

Aplicación. Cada servicio tiene depósitos de recursos separados por dependencia: un depósito de conexiones para PostgreSQL, otro para Redis, otro por cada adaptador externo. En Kubernetes, cada servicio tiene límites propios de CPU y memoria.

Razonamiento frente al ASR. El ASR-3.1 exige disponibilidad de 99,9% con recuperación en menos de 30 segundos ante la caída de Redis. Sin aislamiento, la caída de Redis propaga la saturación a las conexiones de PostgreSQL y la recuperación deja de depender solo de Redis. Con aislamiento, el fallo queda confinado y la ruta de respaldo hacia PostgreSQL tiene recursos disponibles para operar.

Contrapartida aceptada. Se desperdicia capacidad: recursos reservados para una dependencia inactiva no se prestan a otra. Es el costo directo de que un fallo no se propague.

3.10 P5 — Puertos y adaptadores

Problema. El ASR-2.1 exige integrar una pasarela de pagos nueva en menos de 4 horas-hombre y sin modificar ningún servicio del núcleo. Si el servicio de Pagos conoce el formato de la pasarela, cada pasarela nueva obliga a modificarlo, probarlo y desplegarlo.

Aplicación. El núcleo define un puerto —una interfaz expresada en el lenguaje del dominio: autorizar cobro, confirmar cobro, reversar cobro— y no conoce ninguna implementación. Cada pasarela se integra escribiendo un adaptador que implementa ese puerto y traduce entre el modelo de la pasarela y el del dominio. La selección del adaptador ocurre por configuración según el país. Integrar una pasarela nueva es escribir una clase nueva y agregar una entrada de configuración: cero archivos modificados en el núcleo.

Razonamiento frente al ASR. La medida de respuesta del ASR-2.1 —«cero cambios en servicios del núcleo»— es una afirmación verificable sobre el diagrama de dependencias, no sobre la buena voluntad del equipo. La inversión de dependencias es lo que la vuelve estructuralmente cierta: el núcleo no puede depender de una pasarela porque la flecha de dependencia apunta al revés.

Contrapartida aceptada. Hay una capa de indirección adicional y un modelo de dominio que mantener en paralelo a los modelos de los proveedores. Para un sistema con un solo proveedor sería sobre costo injustificado; con cinco categorías de proveedores externos y expansión a cuatro países, se paga solo.

3.11 P6 — Estrategia con configuración externalizada

Problema. El ASR-2.2 exige lanzar un ramo nuevo en dos semanas sin tocar la lógica central del motor de rating. Un motor con las reglas de cada ramo escritas en código requiere modificar, probar y desplegar el motor por cada ramo.

Aplicación. El motor de rating no conoce ramos: conoce cómo evaluar un árbol de reglas y cómo componer factores. Las reglas de cada ramo, sus factores, coberturas y parámetros viven en configuración versionada, no en el código. Añadir el seguro de mascota es cargar una definición nueva. Lo mismo aplica a la regionalización: los parámetros de México son una configuración, no una bifurcación del código.

Razonamiento frente al ASR. El objetivo de negocio de expandirse a tres países en 36 meses y el ASR-2.2 apuntan a la misma propiedad estructural: la variabilidad del negocio debe estar en los datos y no en el código. Si está en el código, cada variación cuesta un ciclo de despliegue y el plazo de dos semanas se consume en el proceso, no en el trabajo.

Contrapartida aceptada. Una configuración mal formada puede romper el motor en ejecución, un error que un compilador habría detectado. Se mitiga validando la definición contra un esquema al cargarla y ejecutando un juego de casos de referencia antes de activarla.

3.12 P7 — Backend por canal

Problema. Web y móvil tienen necesidades opuestas. Web puede recibir cargas útiles grandes en una sola petición; móvil necesita cargas pequeñas, tolerancia a red intermitente y un protocolo de sincronización. Un backend único termina sirviendo mal a ambos.

Aplicación. Dos backends: el BFF Web compone vistas ricas agregando varios servicios del núcleo en una respuesta; el BFF Móvil entrega cargas mínimas, gestiona el protocolo de sincronización con el almacenamiento local del dispositivo y resuelve conflictos.

Razonamiento frente al ASR. El ASR-3.3 exige que el 100% de las consultas de póliza funcionen sin conexión y que la sincronización tome 10 segundos o menos. Eso requiere lógica de sincronización con estado, versiones y resolución de conflictos, que no tiene ningún sentido en el canal web. Ponerla en un backend compartido obligaría al canal web a cargar con complejidad que no usa.

Contrapartida aceptada. Hay lógica de composición duplicada entre los dos BFF. Se acepta porque la alternativa —un backend que sirve a ambos— acopla la evolución de los dos canales: cada cambio para móvil obliga a re-probar web.

3.13 P8 — Publicación y suscripción de eventos de dominio

Problema. La emisión de una póliza dispara efectos en varios servicios: notificar al cliente, actualizar la administración de pólizas, registrar auditoría, alimentar tableros regulatorios. Si Emisión llama sincrónicamente a los cuatro, su latencia es la suma de las cuatro y su disponibilidad el producto de las cuatro.

Aplicación. Emisión publica un evento de dominio y termina. Los servicios interesados se suscriben y reaccionan a su ritmo. El bus retiene los eventos 7 días, lo que permite reprocesar cuando un consumidor estuvo caído o cuando se corrige un error de procesamiento.

Razonamiento frente al ASR. Sostiene el ASR-1.2 al sacar del camino crítico todo lo que no es necesario para responderle al usuario, y refuerza el ASR-2.1: agregar un consumidor nuevo —por ejemplo, un tablero regulatorio para un país nuevo— no requiere tocar el productor.

Contrapartida aceptada. Consistencia eventual. Durante un intervalo breve la póliza existe pero el tablero aún no la refleja. Es aceptable para efectos secundarios; no lo sería para el cobro de la prima, que por eso permanece sincrónico dentro de la transacción de emisión.

3.14 P9 — Tokenización de datos sensibles

Problema. El ASR-4.1 exige cero datos personales en texto plano en tránsito, y la regulación exige poder revocar un consentimiento con efecto en menos de 5 minutos. Si el dato personal está copiado en las bases de siete servicios, revocar significa borrarlo en siete lugares y demostrarlo.

Aplicación. Un único servicio custodia los datos sensibles. El resto del sistema opera sobre tokens que no tienen valor si se filtran. Un servicio que necesita el dato original debe solicitarlo presentando autorización, y cada solicitud queda registrada. Revocar un consentimiento es invalidar la capacidad de destokenizar, lo que surte efecto de inmediato en todo el sistema sin tocar siete bases de datos.

Razonamiento frente al ASR. Convierte la afirmación del ASR-4.1 en algo demostrable: para verificar que no hay información personal en texto plano basta con inspeccionar el tráfico y los volcados de las bases y comprobar que solo aparecen tokens. Con el dato replicado, la verificación tendría que ser exhaustiva sobre todo el sistema.

Contrapartida aceptada. El servicio de tokenización es un punto único de falla y un salto adicional de latencia en las operaciones que requieren el dato original. Se mitiga porque el camino crítico de cotización opera sobre el perfil derivado y no necesita destokenizar.

3.15 P10 — Almacenamiento inmutable con firma digital

Problema. El ASR-4.2 exige detectar en menos de 1 segundo cualquier alteración no autorizada de una póliza emitida, incluyendo alteraciones hechas con credenciales internas comprometidas. Un control basado en permisos no sirve: se asume que el atacante ya tiene los permisos.

Aplicación. Al emitirse, cada póliza se firma con una llave gestionada y se almacena en un contenedor con bloqueo de escritura que impide la modificación y el borrado durante el periodo de retención, incluso para cuentas administrativas. Cada lectura verifica la firma. Cada escritura genera un registro de auditoría inmutable con el servicio, el momento y la huella antes y después.

Razonamiento frente al ASR. El control no depende de que el atacante carezca de permisos, sino de que la plataforma de almacenamiento no acepte la operación. Esto cambia la naturaleza de la garantía: pasa de ser una política a ser una propiedad del sistema.

Contrapartida aceptada. El periodo de retención inmutable implica costo de almacenamiento que no se puede liberar anticipadamente, y errores legítimos no se corrigen borrando sino emitiendo un documento compensatorio. Es exactamente el comportamiento que la regulación espera de una aseguradora.

3.16 P11 — Cliente con prioridad al modo desconectado

Problema. El ASR-3.3 exige que el 100% de las consultas de póliza estén disponibles sin conexión. Un cliente que consulta al servidor y muestra un error cuando no hay red no puede cumplirlo, por rápido que sea el servidor.

Aplicación. La app trata el almacenamiento local cifrado como su fuente de lectura y la red como un mecanismo de actualización en segundo plano. Las pólizas se sincronizan al iniciar sesión y se conservan con vigencia de 24 horas. Las acciones ejecutadas sin conexión se encolan y se envían en orden al recuperar conectividad.

Razonamiento frente al ASR. La medida «100% de consultas disponibles sin conexión» solo es alcanzable si la ausencia de red es el caso normal y no la excepción. Esta es una decisión de diseño del cliente que no se puede añadir después: reordena de dónde lee la aplicación.

Contrapartida aceptada. Datos potencialmente desactualizados, resolución de conflictos y datos sensibles en el dispositivo. Se mitiga con vigencia de 24 horas, aviso visible al vencerse, precedencia del servidor ante conflicto y cifrado del almacenamiento local limitado a lo mínimo necesario para mostrar la póliza.

3.17 P12 — Autoescalado con comprobaciones de salud

Problema. El ASR-3.2 exige RTO de 10 minutos ante la pérdida de una zona. Si la recuperación depende de arrancar instancias nuevas, el tiempo lo determina el arranque en frío.

Aplicación. Los pods se distribuyen entre zonas con reglas de antiafinidad, de modo que ninguna zona concentra todas las réplicas de un servicio. Cada pod expone comprobaciones de vitalidad y de disponibilidad, y el balanceador retira de rotación al que no responde. El autoescalador horizontal reacciona al uso de CPU con umbral del 70% y ventana de 30 segundos, con reducción lenta para evitar oscilaciones.

Razonamiento frente al ASR. Al mantener capacidad activa en ambas zonas, la recuperación se reduce al tiempo de detección más el de redistribución, ambos del orden de decenas de segundos. El autoescalado cubre el segundo efecto de perder una zona: la zona superviviente recibe el doble de tráfico y debe crecer.

Contrapartida aceptada. Se paga capacidad ociosa en operación normal, del orden del doble de la estrictamente necesaria. Es el precio explícito de un RTO de minutos.

4. Decisiones de arquitectura y alternativas descartadas

Documentar lo que se descartó y por qué es tan informativo como documentar lo elegido.

#	Decisión adoptada	Alternativas evaluadas	Razón del descarte
D1	Microservicios por contexto de dominio	Monolito modular	El monolito habría simplificado el desarrollo en 8 semanas, pero impide escalar de forma independiente el servicio de scoring, que es el único con exigencia de latencia de 200 ms y el único con picos de 10×
D2	PostgreSQL como almacenamiento transaccional	Base documental como primaria	Restricción dura del proyecto; además la emisión requiere transacciones con garantías fuertes entre póliza, pago y consentimiento
D3	Kafka como bus de eventos	Cola de mensajes tradicional	La retención y la relectura de eventos son necesarias para reprocesar y para incorporar consumidores nuevos sin coordinación con el productor
D4	Dos BFF, uno por canal	BFF único compartido	La lógica de sincronización offline es específica de móvil; compartirla acopla la evolución de ambos canales
D5	Tokenización centralizada	Cifrado a nivel de campo en	El cifrado por campo replica

#	Decisión adoptada	Alternativas evaluadas	Razón del descarte
		cada servicio	el dato en siete bases y hace que revocar un consentimiento en menos de 5 minutos sea inverificable
D6	Multi-AZ activo-activo	Activo-pasivo con recuperación	Un esquema pasivo hace que el RTO dependa del arranque en frío, lo que pone en riesgo el objetivo de 10 minutos del ASR-3.2
D7	Kotlin nativo para móvil	Framework multiplataforma	El acceso a biometría, cámara con validación de nitidez y almacenamiento seguro del dispositivo es de primera clase en nativo; el onboarding biométrico es el recorrido de mayor riesgo técnico
D8	Angular para web	Alternativa basada en biblioteca	Decisión del equipo del 19 de agosto; el marco de trabajo completo aporta enrutamiento, formularios e internacionalización integrados, relevantes para los cuatro locales objetivo

5. Programa de experimentación de arquitectura

5.1 EXP-01 — Viabilidad del presupuesto de latencia del motor de scoring

Campo	Contenido
Título	Validación de latencia del motor de scoring en condiciones normales de operación
Propósito del experimento	Determinar si la decisión de resolver el scoring desde un perfil derivado cacheado hace viable el presupuesto de 200 ms, y medir el reparto real de ese presupuesto entre sus componentes
Resultados esperados	p95 < 200 ms y p99 < 400 ms bajo 500 solicitudes por minuto, con tasa de acierto de caché ≥ 80% y 0% de errores
Recursos requeridos	Servicios de Cotización & Rating y de Perfilamiento, Redis, PostgreSQL, adaptador Open Finance, WireMock para simular el proveedor y Apache JMeter para generar carga
Elementos de arquitectura	ASR-1.1. Vista funcional: Cotización & Rating, Perfilamiento, Adaptador Open Finance, Redis. Vista de despliegue: EKS, RDS PostgreSQL, ElastiCache

Campo	Contenido
Esfuerzo estimado	10 horas-hombre

Hipótesis de diseño

Campo	Contenido
Puntos de sensibilidad	Tasa de acierto del caché de perfiles y reparto del presupuesto de latencia entre saltos
Historias de arquitectura asociadas	SOL-28 (HU-ARQ-07) · SOL-26 (HU-ARQ-05)
Nivel de incertidumbre	Alta. El equipo no sabe qué tasa de acierto produce una vigencia de 15 minutos en un flujo donde el usuario itera sobre planes, ni cuánto del presupuesto consume el procesamiento propio. Si el rating consume 150 ms, el diseño no cierra por más que el caché acierte
Patrones de arquitectura	Cache-Aside
Descripción de los patrones	El servicio consulta primero el caché y solo ante ausencia recurre a la fuente costosa, escribiendo el resultado con vigencia acotada
Tácticas de arquitectura	Mantener múltiples copias de los datos computados · Reducir la demanda computacional en el camino crítico · Acotar el tiempo de ejecución mediante presupuesto por salto
Descripción de las tácticas	Se evita la llamada externa en el camino crítico sirviendo el perfil derivado desde memoria, y se asigna a cada salto un tiempo máximo derivado hacia atrás del objetivo extremo a extremo

Componentes involucrados

Componente	Propósito y comportamiento esperado	Tecnología
Cotización & Rating	Recibe la solicitud, pide el perfil, aplica el motor de rating y compone la prima. Debe consumir menos de 40 ms en el cálculo	Python 3.11 · Flask · Gunicorn
Perfilamiento & Scoring	Resuelve el perfil consultando primero el caché. Tasa de acierto $\geq$ 80%, menos de 20 ms en el camino de acierto	Python 3.11 · Flask
Adaptador Open Finance	Traduce del modelo del proveedor al de dominio respetando un tiempo de espera de 150 ms	Python 3.11 · httpx
Caché de perfiles	Sirve el perfil derivado con vigencia de 15 minutos	Redis 7
Base de datos	Provee tarifas y parámetros del ramo	PostgreSQL 15

Conectores involucrados

Conector	Comportamiento a probar	Tecnología
Cotización → Perfilamiento	Que el salto de red añada menos de 5 ms al p95	HTTP/1.1 con conexión persistente y depósito
Perfilamiento → Caché	Que la lectura se resuelva por debajo de 2 ms en p99	Protocolo RESP · redis-py con depósito
Cotización → Base de datos	Que la consulta de tarifas no supere 10 ms	psycopg con depósito
Perfilamiento → Proveedor externo	Que el tiempo de espera de 150 ms se respete y no se desborde	HTTP sobre TLS hacia WireMock con retardo fijo

Medición

Métrica	Instrumento	Umbral
Latencia extremo a extremo por percentil	Apache JMeter	p95 < 200 ms · p99 < 400 ms
Tasa de acierto del caché	Contadores del servicio de perfilamiento	≥ 80%
Latencia atribuida a cada salto	Trazas distribuidas	Ver tabla de conectores
Tasa de error	Apache JMeter	0%

Criterio de refutación. La hipótesis queda refutada si el p95 supera 200 ms **teniendo la tasa de acierto en el objetivo**: eso significaría que el problema no está en la estrategia de caché sino en el procesamiento propio.

Decisión alternativa ante resultado adverso. Se revisa el momento del cálculo: se pasa de calcular la prima de forma sincrónica a precalcular las combinaciones más frecuentes de ramo y perfil, sirviendo el cálculo exacto solo para las poco frecuentes. Convierte el problema de latencia en uno de espacio, más barato de resolver.

Tecnología asociada

Elemento	Detalle
Justificación	Las herramientas permiten generar carga controlada, simular la dependencia externa con retardo determinista y atribuir el tiempo consumido a un componente concreto, que es lo que el experimento necesita medir
Lenguaje	Python 3.11
Framework	Flask · Gunicorn
Plataforma de despliegue	Docker Compose (experimento) · AWS EKS (destino)
Bases de datos	PostgreSQL 15 · Redis 7
Librerías	redis-py · psycopg · httpx
Herramientas de análisis	Apache JMeter 5.6 · WireMock · trazas distribuidas · CloudWatch

5.2 EXP-02 — Calibración de la degradación elegante

Campo	Contenido
Título	Validación de degradación elegante ante lentitud del

Campo	Contenido
	proveedor Open Finance
Propósito del experimento	Encontrar la calibración del interruptor de circuito que abre a tiempo para evitar el agotamiento de recursos sin abrir de forma espuria, y confirmar que la ruta degradada sostiene el objetivo de latencia
Resultados esperados	Interruptor abierto en menos de 10 s desde el inicio de la degradación, p95 < 200 ms sirviendo desde caché, 0% de solicitudes perdidas por timeout externo y 0 aperturas espurias en operación normal
Recursos requeridos	Montaje del EXP-01 más el interruptor de circuito instrumentado, WireMock con inyección de retardo creciente y Apache JMeter
Elementos de arquitectura	ASR-1.2. Vista funcional: Perfilamiento, Adaptador Open Finance, Redis, Event Bus
Esfuerzo estimado	12 horas-hombre

Hipótesis de diseño

Campo	Contenido
Puntos de sensibilidad	Umbral y ventana del interruptor de circuito · reparto del presupuesto de latencia entre tiempos de espera
Historias de arquitectura asociadas	SOL-37 (HU-ARQ-08) · SOL-28 (HU-ARQ-07) · SOL-26 (HU-ARQ-05)
Nivel de incertidumbre	Alta. El interruptor es un mecanismo conocido, pero sus parámetros no se derivan analíticamente. Un umbral alto deja que el sistema se sature antes de abrir; uno bajo abre ante fluctuaciones normales y degrada sin necesidad. El equipo no tiene datos previos para elegirlos
Patrones de arquitectura	Circuit Breaker · Fallback
Descripción de los patrones	El interruptor corta las llamadas a una dependencia degradada tras superar un umbral de fallos, y el fallback continúa la operación con la fuente alternativa en lugar de propagar el error
Tácticas de arquitectura	Detección de fallas mediante monitoreo de la tasa de respuesta · Degradación elegante ante indisponibilidad · Acotar el tiempo de espera para liberar recursos
Descripción de las tácticas	Se limita el tiempo que el sistema espera a un tercero y se decide dejar de esperar antes de que el depósito de conexiones se agote

Componentes involucrados

Componente	Propósito y comportamiento esperado	Tecnología
Perfilamiento & Scoring	Aloja el interruptor y decide entre ruta normal y degradada. Debe	Python 3.11 · pybreaker

Componente	Propósito y comportamiento esperado	Tecnología
	conmutar en menos de 10 s	
Adaptador Open Finance	Recurso protegido por el interruptor. Debe fallar rápido sin consumir conexiones cuando está abierto	Python 3.11 · httpx
Cotización & Rating	Consume el perfil sin saber por qué ruta vino. Mantiene p95 < 200 ms durante la degradación	Python 3.11 · Flask
Caché de perfiles	Sostiene el volumen completo de tráfico redirigido	Redis 7

Conectores involucrados

Conector	Comportamiento a probar	Tecnología
Perfilamiento → Proveedor externo	Que ante latencia superior a 700 ms el interruptor abra antes de agotar el depósito	httpx con pybreaker y tiempo de espera de 150 ms
Perfilamiento → Caché	Que la ruta degradada absorba el 100% del tráfico redirigido	redis-py con depósito dimensionado para el pico
Instrumentación del interruptor	Que las transiciones de estado se registren con marca de tiempo precisa	Métricas expuestas por el servicio
Servicios → Event Bus	Que la publicación de eventos no bloquee la respuesta al usuario	Kafka 3.x (KRaft) · kafka-python

Medición

Métrica	Instrumento	Umbral
Tiempo hasta la apertura del interruptor	Métrica de estado del interruptor	< 10 s
Latencia durante la degradación	Apache JMeter	p95 < 200 ms
Solicitudes perdidas por agotamiento de recursos	JMeter · métricas del depósito	0%
Aperturas espurias bajo carga nominal	Métrica de estado del interruptor	0 en 10 minutos

Criterio de refutación. Queda refutada si ninguna configuración de la rejilla ensayada logra abrir a tiempo sin producir aperturas espurias: eso indicaría que el problema no es de calibración sino de dimensionamiento de los depósitos de recursos.

Decisión alternativa ante resultado adverso. Se adopta el mamparo de aislamiento como mecanismo primario en lugar de complementario: se asigna al adaptador Open Finance un depósito de conexiones propio y estrecho, de modo que su saturación sea imposible de propagar aunque el interruptor no abra a tiempo.

Tecnología asociada

Elemento	Detalle
Justificación	Se requiere inyectar retardo de forma progresiva y observar el estado interno del interruptor, lo que exige

Elemento	Detalle
	instrumentarlo y no solo medir desde fuera
Lenguaje	Python 3.11
Framework	Flask · Unicorn
Plataforma de despliegue	Docker Compose · AWS EKS
Bases de datos	Redis 7 · PostgreSQL 15
Librerías	pybreaker · httpx · redis-py · kafka-python
Herramientas de análisis	Apache JMeter 5.6 · WireMock con retardo variable · Toxiproxy

5.3 EXP-03 — Costo y hermeticidad de la tokenización de datos sensibles

Campo	Contenido
Título	Validación de confidencialidad y protección de información financiera Open Finance
Propósito del experimento	Medir el costo en latencia de la tokenización centralizada y verificar empíricamente que no existe ninguna ruta por la cual el dato original escape del servicio custodio
Resultados esperados	100% de las transmisiones sobre TLS 1.3, 100% de los campos personales tokenizados antes de salir de Perfilamiento, cero datos personales en texto plano en tránsito ni en reposo, y latencia añadida por la tokenización inferior a 15 ms
Recursos requeridos	Servicio de Tokenización, Perfilamiento, Consentimientos, adaptador Open Finance, AWS KMS, certificados TLS, OWASP ZAP, captura de tráfico y datos cebo
Elementos de arquitectura	ASR-4.1. Vista funcional: Tokenización, Perfilamiento, Consentimientos & Auditoría, Adaptador Open Finance. Vista de información: clasificación de datos restringidos
Esfuerzo estimado	12 horas-hombre

Hipótesis de diseño

Campo	Contenido
Puntos de sensibilidad	Centralizar la custodia del dato sensible en un único servicio en lugar de cifrar por campo en cada servicio
Historias de arquitectura asociadas	SOL-23 (HU-ARQ-03) · SOL-22 (HU-ARQ-04)
Nivel de incertidumbre	Alta. La decisión tiene dos riesgos que no se resuelven en el papel. De rendimiento: el servicio custodio es un salto adicional y un punto único, y no se sabe cuánto añade al camino crítico. De hermeticidad: el diseño afirma que el dato original no sale del custodio, pero esa afirmación solo se sostiene inspeccionando lo que realmente circula y se persiste

Campo	Contenido
Patrones de arquitectura	Tokenization · Defense in Depth
Descripción de los patrones	El dato sensible se sustituye por un token sin valor fuera del sistema y su original queda bajo custodia única; y la protección se aplica en capas, cifrando también el transporte y el reposo
Tácticas de arquitectura	Limitar el acceso mediante custodia centralizada · Cifrar los datos en tránsito y en reposo · Registrar la auditoría de cada acceso al dato original
Descripción de las tácticas	Revocar un consentimiento se convierte en invalidar la capacidad de destokenizar, lo que surte efecto de inmediato en todo el sistema sin tocar siete bases de datos

Componentes involucrados

Componente	Propósito y comportamiento esperado	Tecnología
Tokenización	Custodia el dato original y entrega tokens sin valor fuera del sistema. Tokeniza en menos de 15 ms y registra cada destokenización	Python 3.11 · Flask · cifrado con llave gestionada
Gestión de llaves	Custodia y rota las llaves de cifrado; ningún servicio accede al material de la llave	AWS KMS
Perfilamiento & Scoring	Opera solo sobre el perfil derivado; no solicita destokenización en el camino crítico	Python 3.11 · Flask
Consentimientos & Auditoría	Registra el consentimiento y su revocación, con efecto en menos de 5 minutos	Python 3.11 · Flask · PostgreSQL 15
Adaptador Open Finance	Negocia TLS 1.3 con el proveedor y entrega los datos ya tokenizados aguas adentro	Python 3.11 · httpx

Conectores involucrados

Conector	Comportamiento a probar	Tecnología
Servicio a servicio	Que el 100% de las conexiones negocie TLS 1.3 y ninguna caiga a una versión anterior	TLS 1.3 entre contenedores
Perfilamiento → Tokenización	Que el salto añada menos de 15 ms y no esté en el camino crítico de cotización	HTTP sobre TLS · httpx con depósito
Tokenización → Gestión de llaves	Que el material de la llave nunca salga del servicio de llaves	AWS KMS · boto3
Servicios → Bus de eventos	Que ningún evento publicado contenga datos personales en texto plano	Kafka 3.x · consumidor de inspección

Conector	Comportamiento a probar	Tecnología
Servicios → Base de datos	Que ninguna tabla persista datos personales en texto plano	psycopg · inspección de volcados

Medición

Métrica	Instrumento	Umbral
Coincidencias de datos cebo fuera del custodio	Búsqueda sobre volcados, tráfico capturado y mensajes del bus	0
Versión de TLS negociada	OWASP ZAP · captura de tráfico	TLS 1.3 en el 100%
Latencia añadida por la tokenización	Trazas distribuidas	< 15 ms
Entradas de auditoría por destokenización	Consulta al registro de auditoría	Una por acceso, sin excepción
Efecto de la revocación de consentimiento	Prueba de extremo a extremo cronometrada	< 5 minutos

Criterio de refutación. Una sola coincidencia de dato cebo en un volcado, en un mensaje del bus o en el tráfico refuta la hipótesis e identifica el punto exacto de fuga, que es precisamente el valor del experimento.

Decisión alternativa ante resultado adverso. Si aparece una fuga por el bus de eventos, se adopta la publicación de eventos con referencia en lugar de con contenido: el evento transporta un identificador y el consumidor recupera lo que necesite sujeto a su propia autorización. Si la latencia añadida resulta inaceptable, se introduce un caché de tokens de corta vigencia en el consumidor, aceptando el aumento de superficie de exposición.

Tecnología asociada

Elemento	Detalle
Justificación	La hermeticidad no se demuestra leyendo código: exige sembrar datos reconocibles y buscarlos en todo lo que el sistema transmite y persiste. De ahí el uso de datos cebo y de inspección directa de volcados y temas
Lenguaje	Python 3.11
Framework	Flask
Plataforma de despliegue	Docker Compose · AWS EKS
Bases de datos	PostgreSQL 15 · Redis 7
Librerías	cryptography · boto3 · httpx · psycopg · kafka-python
Herramientas de análisis	OWASP ZAP · captura de tráfico · consumidor de inspección de Kafka · consultas de verificación sobre PostgreSQL · datos cebo

5.4 Alternativas evaluadas y descartadas

Se diseñaron y evaluaron cinco experimentos adicionales que finalmente no se proponen. Documentar por qué se descartaron es parte del criterio de estimación: no se trata de no haberlos pensado.

Candidato	ASR	Por qué se descarta
Caída del caché — verificar que la ruta de respaldo hacia PostgreSQL absorba el tráfico sin propagar la saturación	ASR-3.1	Se cubre parcialmente en el EXP-02: el interruptor de circuito y el mamparo que se calibran ahí son los mismos mecanismos que operan

Candidato	ASR	Por qué se descarta
		ante la caída de Redis. Se instrumenta el EXP-02 para registrar también ese escenario, en lugar de montar un experimento propio. Ahorra 8 horas
Modificabilidad de los puntos de extensión — medir el costo de integrar una pasarela nueva	ASR-2.1 · ASR-2.2	Su medida es el conjunto de archivos que cambian, y eso se comprueba revisando el grafo de dependencias y el registro del control de versiones. No requiere montaje ni ejecución: es una verificación estática que se hace durante la revisión de código
Pérdida de una zona de disponibilidad	ASR-3.2	Requiere infraestructura multi-zona en la nube con costo real y un servicio de inyección de fallas. El presupuesto de esta fase no lo cubre. Se difiere a la fase de construcción, donde la infraestructura ya estará provisionada
Continuidad del cliente móvil sin conexión	ASR-3.3	El prototipo móvil que se construye como parte de la experiencia de usuario ya ejercita este escenario. Validarlo ahí evita duplicar el montaje del emulador
Integridad de pólizas emitidas	ASR-4.2	El bloqueo de escritura del almacenamiento de objetos es una garantía documentada de la plataforma y sobre eso el equipo no tiene incertidumbre. Lo que sí es incierto —el tiempo de detección de la ruta de auditoría propia— se verifica con una prueba de configuración dentro de la suite de seguridad, no con un experimento

5.5 Ficha consolidada de tecnología

Categoría	Tecnología	Experimentos
Lenguaje backend	Python 3.11	Todos
Lenguaje móvil	Kotlin	Prototipo móvil
Lenguaje web	TypeScript · Angular 17	EXP-03
Framework backend	Flask · Gunicorn	Todos
Base de datos	PostgreSQL 15	01, 03, 04, 05, 07, 08
Caché	Redis 7	01, 02, 04, 07
Bus de eventos	Kafka 3.x (KRaft) · Amazon MSK	02, 05, 07
Almacenamiento de objetos	Amazon S3 con Object Lock	08
Gestión de llaves	AWS KMS	07, 08

Categoría	Tecnología	Experimentos
Cliente de base de datos	psycopg con depósito	01, 03, 04, 05, 07
Cliente de caché	redis-py con depósito	01, 02, 04
Cliente HTTP	httpx con depósito y tiempo de espera	01, 02, 03, 07
Interruptor de circuito	pybreaker	02, 04
Cliente de eventos	kafka-python	02, 05, 07
SDK de nube	boto3	07, 08
Criptografía	cryptography	07, 08
Validación de esquema	jsonschema	03
Cliente móvil	Retrofit · OkHttp · WorkManager · Room con SQLCipher	Prototipo móvil
Orquestación local	Docker · Docker Compose	Todos
Orquestación de nube	Amazon EKS · Horizontal Pod Autoscaler	05
Generación de carga	Apache JMeter 5.6	01, 02, 04, 05
Simulación de dependencias	WireMock	01, 02
Inyección de fallas	Toxiproxy · docker stop · AWS Fault Injection Service	02, 04, 05
Trazas distribuidas	Instrumentación de trazas por salto	01, 02, 07
Seguridad	OWASP ZAP	07
Pruebas backend	pytest 8 · pytest-cov	03 y pruebas residuales
Pruebas móviles	Espresso · JUnit 5 · MockK	06
Pruebas de API	Postman · Newman	03
Observabilidad	AWS CloudWatch · CloudTrail	04, 05, 08
Control de versiones	Git con registro de cambios por archivo	03